

A PROJECT ON

Decentralized Electronic Health Records (EHR) System Using Blockchain

**Submitted in partial fulfillment of the requirement for the award of the
degree of**

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE & ENGINEERING (Write Appropriate Program Name)
Submitted by:**

Student Name 1 : Aayush Garg	University Roll No : 2021943
Student Name 2 : Akshit Ramola	University Roll No : 2021936
Student Name 3 : Aayush Bhardwaj	University Roll No : 2021941
Student Name 4 : Nikhil Bhatt	University Roll No : 2021962

Under the Guidance of
Dr. Pradeep Bedi
Associate Professor

**Project Team ID:
MP2025IOT15**



**Department of Computer Science and Engineering
Graphic Era (Deemed to be University)
Dehradun, Uttarakhand
May-2026**



DECLARATION

We hereby certify that the work which is being presented in the project progress report entitled “**Decentralized Electronic Health Records (EHR) System Using Blockchain**” in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (**IOT**) in the Department of Computer Science and Engineering of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision of **Dr. Pradeep Bedi, Associate Professor**, Department of Computer Science and Engineering, Graphic Era (Deemed to be University), Dehradun.

The matter presented in this project report has not been submitted by us for the award of any other degree at this or any other institute/university.

Name : Aayush Garg	2021943	signature
Name : Akshit Ramola	2021936	signature
Name : Ayush Bhardwaj	2021941	signature
Name : Nikhil Bhatt	2021962	signature

This is to certify that the above statement made by the candidate is correct to the best of our knowledge.

Signature

Supervisor

Signature

Head of the Department

External Viva

Name of the Examiners:

Signature with Date

- 1.
- 2.

CERTIFICATE

On the basis of the declaration submitted by Aayush Garg, Akshit Ramola, Aayush Bhardwaj, Nikhil Bhatt, students of B.Tech CSE, we hereby certify that the project titled “**Decentralized Electronic Health Records (EHR) System Using Blockchain**” which is submitted to the Department of Computer Science and Engineering, Graphic Era (Deemed to be University), Dehradun, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is an original contribution with existing knowledge and a faithful record of work performed under my guidance and supervision.

To the best of my knowledge, this work has not been submitted in part or in full for any Degree or Diploma elsewhere.

Dehradun

Date:

Name of the Supervisor

Designation

Department of Computer Science and Engineering

Graphic Era (Deemed to be University),

Dehradun, Uttarakhand, India.

ACKNOWLEDGEMENT

I take this opportunity to express my profound sense of gratitude and respect to all those who helped me throughout our project.

This report acknowledges the intense driving and technical competence of the entire individual that have contributed to it. It would have been almost impossible to complete this project without the support of these people. I extend thanks and gratitude to **Prof. (Dr.) Devesh Pratap Singh** , HOD, Department of Computer Science and Engineering, and **Dr.Pradeep Bedi, Associate Professor**, Department of Computer Science and Engineering, who have imparted to me guidance in all aspects. They shared their valuable time from their busy schedule to guide me and provide their active and sincere support for my activities.

This report is an authentic record of my own work, which is accomplished by the sincere and active support of all the teachers of my college. I/we have tried my best to summarize this report.

Name1 : Aayush Garg	University Roll no1:2021943	signature
Name2 : Akshit Ramola	University Roll no2:2021936	signature
Name3 : Aayush Bhardwaj	University Rollno3: 2021941	signature
Name4 : Nikhil Bhatt	University Roll no4: 2021962	signature

Abstract

The management of Electronic Health Records (EHR) in traditional centralized healthcare systems suffers from critical issues including data breaches, unauthorized access, lack of interoperability, and patient disempowerment. This project presents a Decentralized Electronic Health Record System built on blockchain technology that leverages Ethereum-compatible smart contracts deployed on the Polygon network to ensure tamper-proof, patient-centric management of medical data. The system stores sensitive health documents on the InterPlanetary File System (IPFS) via the Pinata API, with only cryptographic hash references written onto the blockchain, thereby ensuring both data immutability and storage efficiency. The backend is developed using Spring Boot (Java 17) integrated with the Web3j library for blockchain interaction and Spring Security for role-based access control. The frontend is built with React.js and communicates with MetaMask for wallet-based authentication. The proposed system implements role-based access where patients control who can view or update their records, while doctors and hospitals can be granted selective permissions. Smart contracts written in Solidity enforce access policies on-chain, eliminating the need for a trusted third party. The system was tested on the Polygon Mumbai testnet and demonstrated successful registration, record upload, access grant/revocation, and retrieval operations with measurable gas efficiency. The proposed solution offers a scalable, secure, and privacy-preserving alternative to conventional centralized EHR systems, contributing toward the future of sovereign digital health infrastructure

Table of Contents

Contents	Page No.
Declaration	ii
Certificate	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Tables	vii
List of Figures	viii
List of Abbreviation	ix
CHAPTER 1: INTRODUCTION	1-5
1.1 Project Introduction	1
1.2 Problem Statement	3-4
1.3 Objectives	4-5
CHAPTER 2: LITERATURE REVIEW	6-11
CHAPTER 3: PROPOSED METHODOLOGY	12-16
CHAPTER 4: PROJECT DESIGN AND TESTING	17-20
CHAPTER 5: RESULT AND DISCUSSION	21-24
CHAPTER 6: CONCLUSION AND FUTURE SCOPE	25-27
 DETAILS OF RESEARCH PUBLICATION	
 APPENDIX	
 REFERENCES	

List of Tables

2.1	Summary of Existing Research Work on Decentralized EHR Systems (2009-2024)	13
3.1	Technology Stack Used in the Proposed System	21
3.2	Smart Contract Functions and Their Descriptions	25
4.1	Test Cases for Patient Registration Module	30
4.2	Test Cases for Record Upload and Retrieval Module	33
4.3	Gas Consumption Analysis for Smart Contract Functions	36
5.1	Comparison of Proposed System with Existing Centralized Systems	44

List of Figures

FIGURE No.	TITLE	PAGE No.
1.1	Traditional Centralized EHR Architecture	2
1.2	Proposed Decentralized EHR Architecture	4
3.1	Overall System Architecture Diagram	22
3.2	IPFS Data Storage Flow	23
3.3	Solidity Smart Contract Interaction Flow	24
3.4	Web3j and Spring Boot Integration Architecture	26
4.1	Use Case Diagram of the Proposed System	29
4.2	Sequence Diagram for Record Upload	31
4.3	Deployment Diagram on Polygon Mumbai Testnet	35

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
EHR	Electronic Health Record
EMR	Electronic Medical Record
IPFS	InterPlanetary File System
API	Application Programming Interface
JSON	JavaScript Object Notation
REST	Representational State Transfer
PoS	Proof of Stake
PoW	Proof of Work
DApp	Decentralized Application
ABI	Application Binary Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
SHA	Secure Hash Algorithm
P2P	Peer-to-Peer
UI	User Interface
VM	Virtual Machine
EVM	Ethereum Virtual Machine
SDK	Software Development Kit
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment

CHAPTER 1

INTRODUCTION

1.1 PROJECT INTRODUCTION

Healthcare is one of the most sensitive domains where data integrity, privacy, and availability are of paramount importance. Over the past two decades, the digitization of health records has brought about tremendous improvements in the efficiency and accessibility of medical information. Electronic Health Records (EHR) have replaced paper-based medical records and enabled healthcare providers to access patient data instantly from any geographical location. However, despite this digital transformation, the majority of existing EHR systems continue to rely on centralized architectures that present significant vulnerabilities including single points of failure, data breaches, vendor lock-in, and lack of patient control over personal health information.

Centralized EHR systems store all patient data on servers managed by hospitals, clinics, or healthcare service providers. These servers are prime targets for cyberattacks. According to various cybersecurity reports, the healthcare sector consistently ranks among the most targeted industries for data breaches. A single breach can compromise the private health information of millions of patients, leading to identity theft, insurance fraud, and erosion of trust between patients and healthcare providers. Additionally, centralized systems often create interoperability challenges where a patient's medical history recorded at one hospital is not easily accessible to a different hospital due to disparate data formats and siloed databases.

Blockchain technology, first introduced in 2008 as the underlying infrastructure for Bitcoin, has since evolved beyond its financial applications and is now being explored in sectors such as supply chain management, digital identity verification, voting systems, and most recently, healthcare. The core properties of blockchain, namely decentralization, immutability, transparency, and cryptographic security, make it an ideal technological foundation for building a next-generation EHR system that addresses the limitations of centralized architectures.

A blockchain is a distributed ledger that maintains a continuously growing list of records, called blocks, which are linked and secured using cryptographic hashes. Each block contains a timestamp and a reference to the previous block, making the data tamper-evident. In a blockchain-based EHR system, patient health records are not stored directly on the chain due

to storage limitations and privacy concerns. Instead, health data is stored off-chain on a decentralized storage network such as the InterPlanetary File System (IPFS), and only the cryptographic hash (CID) of the file is recorded on the blockchain. This ensures immutability of the hash while keeping the actual data storage efficient and scalable.

The proposed system in this project integrates several cutting-edge technologies to create a complete decentralized EHR platform. At the core is a Solidity smart contract deployed on the Polygon network, which manages patient registration, access control, and record hash storage. The Polygon network was chosen for its Ethereum Virtual Machine (EVM) compatibility, low transaction fees, and high throughput compared to the Ethereum mainnet. The backend is developed using Spring Boot (Java 17) and communicates with the Polygon blockchain via the Web3j library. The frontend is built using React.js and integrates with MetaMask for wallet-based authentication, allowing patients and doctors to interact with the smart contracts directly from their browsers.

The system supports multiple user roles including Patient, Doctor, Hospital Administrator, and Government Regulator. Each role has well-defined permissions enforced at the smart contract level. A patient can upload their health records, grant or revoke access to specific doctors or institutions, and view a complete audit trail of who has accessed their records and when. Doctors can view records of patients who have granted them access and can append new medical notes or prescriptions with a timestamped blockchain transaction. Hospital administrators can manage institutional access policies, while government regulators can access anonymized aggregate data for public health surveillance without compromising individual privacy.

The use of IPFS for off-chain storage ensures that health records are distributed across a peer-to-peer network without any single point of control. The Pinata API is used as a pinning service to ensure that IPFS content remains available and is not garbage collected. Every file uploaded to IPFS returns a unique Content Identifier (CID) which serves as the permanent address of the file. This CID is then stored on the blockchain along with metadata such as the uploader's address, timestamp, and access permissions. The combination of blockchain immutability and IPFS availability provides a robust foundation for secure and permanent health record management.

Security in the proposed system is enforced at multiple layers. The smart contract implements access control logic using Solidity modifiers and mappings. The Spring Boot backend uses

Spring Security with JWT-based authentication for API-level security. All communication between the frontend and backend occurs over HTTPS, and sensitive keys are managed through environment variables as defined in the .env configuration. Role-based access ensures that only authorized parties can perform specific operations, and every action is recorded on the blockchain for auditability.

1.2 PROBLEM STATEMENT

The existing centralized Electronic Health Record systems suffer from several critical challenges that undermine the security, privacy, and efficiency of healthcare data management. These problems have become increasingly severe with the growing volume of digital health data and the rising sophistication of cyberattacks targeting the healthcare sector.

The first and most significant problem is the lack of data security in centralized systems. When patient data is stored in a centralized database managed by a single healthcare provider or IT vendor, it becomes a high-value target for cybercriminals. A successful attack on such a database can expose the personal and medical information of thousands or even millions of patients. Unlike financial data, health data is particularly sensitive because it cannot be changed once compromised; a patient's medical history, diagnoses, and treatment records are permanent and can have lifelong consequences if misused.

The second problem is the absence of patient ownership and control. In the current paradigm, when a patient visits a hospital and undergoes medical examinations, the resulting records are stored in the hospital's database and are treated as the institution's property. Patients have limited or no direct access to their own health data and must rely on administrative processes to obtain copies of their records. This lack of patient-centric data governance violates the fundamental principle that individuals should have sovereignty over their own health information.

The third problem is the lack of interoperability between different healthcare systems. A patient who receives treatment at multiple hospitals or clinics will have their health records scattered across different institutions, each using different data formats and storage systems. When a patient is referred from one hospital to another, the transfer of medical records is often slow, error-prone, and incomplete, which can lead to misdiagnosis, duplicate testing, and suboptimal treatment decisions. This fragmentation of health data is a systemic inefficiency that costs both time and money while potentially harming patient outcomes.

The fourth problem relates to data integrity. In centralized systems, there is no cryptographic guarantee that stored records have not been tampered with. An administrator or insider with access to the database could potentially alter or delete records without any trace. In legal and insurance contexts, the integrity of medical records is critical, and any doubts about their authenticity can have serious consequences.

The fifth problem is the opacity and lack of auditability of data access. Patients generally have no way of knowing who has accessed their health records, when, and for what purpose. This lack of transparency makes it impossible to detect unauthorized access unless a formal audit is conducted, which itself is a resource-intensive process. The absence of a continuous, verifiable audit log leaves patients vulnerable to privacy violations that may go undetected indefinitely.

The proposed decentralized EHR system directly addresses all five of these problems by leveraging blockchain technology, smart contracts, and decentralized storage to create a secure, patient-controlled, interoperable, tamper-proof, and fully auditable health record management platform.

1.3 OBJECTIVES

The primary objectives of this project are as follows:

1. To design and implement a decentralized Electronic Health Record system using Ethereum-compatible blockchain technology that eliminates the need for a central trusted authority.
2. To develop Solidity smart contracts that enforce patient-centric access control, allowing patients to grant and revoke access permissions to healthcare providers at a granular level.
3. To integrate the InterPlanetary File System (IPFS) via the Pinata API for decentralized off-chain storage of health record files, ensuring availability and tamper-evidence of stored data.
4. To build a Spring Boot backend with Web3j integration that provides a RESTful API layer between the frontend application and the blockchain network, enabling seamless blockchain interaction without requiring end users to understand the underlying technology.
5. To develop a React.js frontend that integrates with MetaMask for wallet-based authentication and provides an intuitive interface for patients, doctors, and administrators to interact with the system.

6. To implement role-based access control across the entire system stack, including smart contract level permissions, backend API security using Spring Security and JWT, and frontend route-level protection.
7. To deploy and test the system on the Polygon Mumbai testnet, evaluating the system in terms of transaction throughput, gas consumption, response time, and security robustness.
8. To compare the proposed system with existing centralized EHR solutions in terms of data security, patient control, interoperability, and auditability, demonstrating the advantages of the decentralized approach.

1.4 ORGANIZATION OF THE REPORT

The rest of this project report is organized as follows. Chapter 2 presents a comprehensive literature review of existing research on blockchain-based EHR systems, analyzing the key contributions, limitations, and research gaps that motivated the proposed approach. Chapter 3 describes the proposed methodology in detail, including the system architecture, the design of smart contracts, the IPFS integration strategy, the backend design, and the frontend architecture. Chapter 4 presents the project design and testing, including use case diagrams, sequence diagrams, test case designs, and gas consumption analysis. Chapter 5 presents the results and discussion, including screenshots of the working system, performance evaluation, and a comparative analysis with existing systems. Chapter 6 concludes the report with a summary of contributions and directions for future work. The appendix contains the complete source code listing, and the references section lists all cited works.

CHAPTER 2

LITERATURE REVIEW

2.1 BACKGROUND

The concept of using distributed ledger technology for health record management has been explored extensively since the publication of the original Bitcoin whitepaper by Satoshi Nakamoto in 2008. The blockchain paradigm shift from financial transactions to data management was a natural progression that attracted significant research attention in the healthcare domain, particularly after the introduction of Ethereum and its smart contract functionality in 2015. Over the past decade, numerous academic papers, pilot projects, and real-world implementations have explored how blockchain can transform the way health data is stored, shared, and controlled.

The early research in this domain was primarily theoretical, proposing frameworks and architectures without practical implementation. As blockchain development tools matured, particularly with the advent of Solidity as a smart contract language and platforms like Truffle and Hardhat for development and testing, researchers began building more concrete proof-of-concept systems. The advent of IPFS in 2015 further enriched the design space by providing a viable decentralized storage solution that could complement blockchain's limited on-chain storage capacity.

More recent research has shifted toward practical considerations such as gas optimization for smart contracts, scalability challenges on the Ethereum mainnet (leading to the exploration of layer-2 solutions like Polygon), privacy-preserving techniques such as zero-knowledge proofs, and the integration of blockchain with Internet of Things (IoT) devices for continuous health monitoring. The following sections provide a detailed review of the key research contributions in chronological order, identifying the strengths and limitations of each approach and building toward the identification of the research gap addressed by the present project.

2.2 REVIEW OF RESEARCH WORK

Azaria et al. (2016) introduced MedRec, one of the earliest and most influential blockchain-based EHR systems. MedRec was built on the Ethereum blockchain and used smart contracts to manage authentication, data ownership, and permission-granting for health records stored on existing EHR databases. The system introduced a novel incentivization model where healthcare providers could earn cryptocurrency tokens by contributing computational resources

to the network, thereby participating as miners. While MedRec was groundbreaking in its vision, it faced limitations including high transaction latency due to Ethereum's proof-of-work consensus, lack of scalability, and reliance on existing hospital databases rather than providing a completely decentralized storage solution.

Eckblaw et al. (2017) extended the MedRec concept by demonstrating a working prototype and evaluating it in a clinical setting. Their work highlighted the feasibility of blockchain-based EHR management and identified the importance of standardized data formats for interoperability. The prototype successfully demonstrated patient-controlled access to records from multiple providers but was limited by the high gas costs of the Ethereum mainnet and the lack of a decentralized storage layer.

Xia et al. (2017) proposed MeDShare, a blockchain-based system specifically targeting data sharing among healthcare cloud service providers. MeDShare used smart contracts to enforce data governance policies and provided a fine-grained access control mechanism. The system addressed inter-organizational data sharing but did not provide a patient-facing interface and assumed that all participating entities were trusted cloud providers, which limits its applicability in consumer-facing scenarios.

Dubovitskaya et al. (2017) presented a framework for sharing oncology data using blockchain, focusing on the specific use case of cancer research data sharing between institutions. Their system demonstrated how blockchain could facilitate secondary use of health data for research purposes while preserving patient privacy through role-based access control. However, the system was highly specialized for oncology data and did not provide a general-purpose EHR management solution.

Jiang et al. (2018) proposed BloCHIE, a blockchain-based platform for Healthcare Information Exchange, which separated health data into two categories: healthcare data that changes frequently and personal health data that is relatively static. Different smart contracts handled each category, optimizing storage and access patterns. BloCHIE demonstrated improved performance compared to earlier systems but still relied on the Ethereum mainnet, which presents scalability and cost challenges for production deployment.

Liang et al. (2018) introduced a mobile health data sharing system that used blockchain for consent management. Their system focused specifically on the patient experience of granting and revoking consent for data sharing, implemented as a mobile application. The research contributed valuable insights into usability considerations for blockchain health systems but

was limited to consent management and did not address the full lifecycle of health record management.

Fan et al. (2018) proposed a decentralized health data sharing platform that combined blockchain with attribute-based encryption to provide fine-grained access control. Their approach allowed data to be encrypted in a way that only users with specific attributes (such as a doctor's specialization) could decrypt it, adding a cryptographic layer of privacy. However, the computational overhead of attribute-based encryption was significant and posed challenges for mobile and resource-constrained devices.

Shen et al. (2019) presented a blockchain-based personal health record system with a focus on privacy preservation using searchable encryption. The system allowed encrypted records to be searched without decrypting them, which is particularly important for research and emergency access scenarios. The work made a significant contribution to privacy-preserving search in blockchain health systems but did not address the decentralized storage challenge.

Agbo et al. (2019) conducted a systematic literature review of blockchain applications in healthcare, analyzing 34 papers published between 2016 and 2018. They identified the key areas of application including EHR management, drug traceability, clinical trial data management, and medical insurance fraud detection. The review highlighted the early-stage nature of the field and the predominance of conceptual or small-scale prototype work, calling for more rigorous evaluation in real-world settings.

Mayer et al. (2019) proposed a blockchain-based system for secure sharing of genomic data, which represents one of the most sensitive categories of health information. Their system used a combination of blockchain for access control and secure multi-party computation for privacy-preserving analysis of genomic data. While highly innovative, the computational requirements of the system made it impractical for general deployment.

Xu et al. (2019) proposed a framework combining blockchain with federated learning for healthcare data analysis. Their system allowed machine learning models to be trained on distributed health data without the data leaving the local hospital systems, with blockchain providing a tamper-proof record of model updates. This work pioneered the intersection of blockchain and privacy-preserving machine learning in healthcare but was not directly applicable to EHR management for individual patients.

Rifi et al. (2017) proposed using the Ethereum blockchain for the management of patients' private data in a French hospital context, demonstrating the applicability of the technology in a real institutional setting. Their work identified regulatory and legal challenges associated with storing health data on a public blockchain and proposed a hybrid architecture with a private blockchain for sensitive data.

Gordon and Catalini (2018) provided a theoretical analysis of blockchain applications in healthcare, examining the potential benefits and limitations from an economic and organizational perspective. They argued that blockchain offers the most value in scenarios where multiple parties need to share data without trusting each other, which is precisely the situation in multi-provider healthcare. Their work provided a strong theoretical foundation for subsequent practical implementations.

Benchoufi and Ravaud (2017) explored the use of blockchain for clinical trial data management, focusing on the problem of consent tracking and data integrity in clinical research. Their work demonstrated that blockchain could provide a verifiable, timestamped record of patient consent that is more robust than traditional paper-based consent processes.

Nakamoto et al. (2019) investigated the use of blockchain for medicine supply chain management alongside EHR integration, showing that blockchain could be used to verify the authenticity of medications prescribed in electronic prescriptions and track their delivery to patients.

2.3 RESEARCH SUMMARY TABLE

Table 2.1 below presents a chronological summary of key research contributions in the field of blockchain-based EHR systems over the past fifteen years.

Table 2.1: Summary of Existing Research Work on Decentralized EHR Systems (2009-2024)

S.No.	Author(s)	Title/System	Approach	Limitations
1	Azaria et al. (2016)	MedRec	Ethereum smart contracts for record access management with mining incentives	High latency, no decentralized storage, PoW limitations
2	Ekblaw et al. (2017)	MedRec Prototype	Clinical prototype of MedRec with multi-provider access demonstration	High gas costs, limited storage solution
3	Xia et al. (2017)	MedShare	Data governance for cloud health providers using blockchain	No patient-facing interface, requires trusted cloud providers
4	Dubovitskaya et al. (2017)	Oncology Data Framework	Blockchain for cancer research data sharing	Domain-specific, not general-purpose EHR
5	Jiang et al. (2018)	BlochIE	Dual-contract system for dynamic and static health data	Ethereum mainnet cost and scalability issues
6	Liang et al. (2018)	Mobile Consent System	Mobile app for blockchain-based consent management	Limited to consent, not full EHR lifecycle
7	Fan et al. (2018)	ABE-Blockchain System	Attribute-based encryption with blockchain for fine-grained access	High computational overhead for mobile devices
8	Shen et al. (2019)	Searchable Encryption PHR	Searchable encryption for privacy-preserving record search	No decentralized storage, complex key management
9	Agbo et al. (2019)	Systematic Review	Literature review of 34 blockchain healthcare papers	Review paper, identifies gaps but no implementation
10	Mayer et al. (2019)	Genomic Data Sharing	Blockchain + MPC for genomic privacy	High computational requirements, limited applicability
11	Xu et al. (2019)	FL-Blockchain Healthcare	Federated learning with blockchain for distributed analysis	Not applicable to individual EHR management
12	Rifi et al. (2017)	French Hospital Pilot	Ethereum for patient data in French hospital	Regulatory challenges, hybrid architecture required

2.4 RESEARCH GAP

The review of existing literature reveals several significant gaps that the present work aims to address. First, the majority of existing systems rely on the Ethereum mainnet, which has well-documented scalability limitations and prohibitively high gas fees for real-world healthcare applications where hundreds of transactions may need to be processed per day. Very few studies have explored the use of layer-2 scaling solutions such as Polygon, which offers dramatically lower transaction costs while maintaining Ethereum's security guarantees. The present project specifically addresses this gap by deploying on the Polygon network.

Second, while several systems propose decentralized storage, few provide a complete, end-to-end implementation that integrates IPFS with a production-grade pinning service such as Pinata to ensure long-term availability of stored records. Many existing systems either store records on centralized servers or assume the existence of an IPFS node without addressing the practical challenge of content persistence.

Third, existing systems largely focus on a single aspect of the EHR ecosystem, such as access control, consent management, or data sharing, without providing a comprehensive full-stack solution. The present work addresses this by integrating smart contract-level access control, secure off-chain storage, a REST API backend with Spring Security, and a React.js frontend with MetaMask integration into a unified, deployable system.

Fourth, the role of a production-quality backend framework such as Spring Boot in bridging the gap between the blockchain layer and end users has been underexplored. Most existing implementations rely on JavaScript-based backends or direct frontend interactions with blockchain nodes, which may not meet the enterprise-grade reliability and security requirements of healthcare systems. The present work contributes a Java-based backend using Web3j that demonstrates how enterprise Java ecosystems can be integrated with blockchain infrastructure.

Finally, the auditability and transparency features of the proposed system, which provide patients with a complete, immutable log of all accesses to their records, address the privacy and accountability gap identified across the reviewed literature. These features are essential for regulatory compliance under frameworks such as HIPAA and GDPR, which require both data protection and documentation of data access.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 SYSTEM ARCHITECTURE

The proposed Decentralized Electronic Health Record System adopts a layered, multi-component architecture that separates concerns across the blockchain layer, the decentralized storage layer, the backend application layer, and the frontend presentation layer. This separation of concerns ensures that each component can be independently maintained, upgraded, or replaced without affecting the overall system integrity. The architecture is designed to be modular, scalable, and secure by default, with each layer enforcing its own security policies while delegating trust decisions to the underlying blockchain infrastructure.

The blockchain layer consists of Solidity smart contracts deployed on the Polygon Mumbai testnet. These contracts serve as the immutable, trustless backbone of the system, storing patient registration data, access control mappings, and IPFS content identifiers. The Polygon network was selected because it provides EVM compatibility with the Ethereum mainnet while offering significantly lower transaction fees (typically less than \$0.01 per transaction compared to several dollars on Ethereum mainnet) and faster block confirmation times (approximately two seconds compared to twelve seconds on Ethereum). The network uses a Proof-of-Stake consensus mechanism, which is both more energy-efficient and more performant than Ethereum's historical Proof-of-Work mechanism.

The decentralized storage layer utilizes the InterPlanetary File System (IPFS), a peer-to-peer hypermedia protocol designed to make the web faster, safer, and more open. Unlike traditional web servers that use location-based addressing (URLs), IPFS uses content-based addressing where each piece of content is identified by its cryptographic hash (CID). When a health record file is uploaded to IPFS through the Pinata API, it receives a unique CID that remains constant regardless of where the file is stored in the network. This CID is then stored on the blockchain, creating a permanent, verifiable link between the on-chain reference and the off-chain content.

The backend application layer is implemented as a Spring Boot microservice running on Java 17. This layer provides a RESTful API that serves as the interface between the frontend application and both the blockchain and IPFS layers. The backend uses the Web3j library to generate typed Java wrappers for the deployed smart contracts, enabling type-safe interaction with blockchain functions without requiring frontend developers to understand the intricacies

of blockchain programming. Spring Security is integrated to handle JWT-based API authentication, while CORS configuration ensures that only authorized frontend origins can access the API endpoints.

The frontend presentation layer is built with React.js and provides a user-friendly interface for all stakeholder roles. MetaMask, a browser extension that functions as an Ethereum wallet, handles cryptographic signing of transactions on the client side. This means that private keys never leave the user's device, which is a fundamental security improvement over systems where the backend manages user keys. The frontend communicates with the backend API over HTTPS for regular data operations and directly with MetaMask for transaction signing.

3.2 TECHNOLOGY STACK

Table 3.1 summarizes the technology stack used in the proposed system.

Table 3.1: Technology Stack Used in the Proposed System

Layer	Technology	Purpose
Blockchain	Polygon (Mumbai Testnet)	EVM-compatible L2 network for smart contract deployment
Smart Contract	Solidity 0.8.x	Business logic, access control, CID storage
Dev Framework	Hardhat	Smart contract compilation, testing, deployment
Decentralized Storage	IPFS + Pinata API	Off-chain storage of health record files
Backend	Spring Boot 3.2.3 (Java 17)	REST API, business logic orchestration
Blockchain Integration	Web3j 4.10.3	Java library for Ethereum/Polygon interaction
Security	Spring Security + JWT	API-level authentication and authorization
Frontend	React.js	User interface for all stakeholder roles
Wallet	MetaMask	Browser-based Ethereum wallet for tx signing
Build Tool	Maven (pom.xml)	Java project management and dependency resolution
HTTP Client	Spring WebFlux	Reactive HTTP client for Pinata API calls

3.3 SMART CONTRACT DESIGN

The core of the proposed system is the EHR smart contract written in Solidity. The contract defines the data structures, state variables, events, modifiers, and functions that govern the behavior of the system on the blockchain. The contract is organized around three primary concepts: patient identity, health record storage, and access control.

Patient identity is represented by a mapping from Ethereum wallet addresses to patient data structures containing the patient's name, date of birth, blood type, emergency contact, and registration timestamp. The patient's wallet address serves as their unique identifier on the blockchain, eliminating the need for username and password authentication. When a patient registers with the system, a transaction is sent to the `registerPatient` function of the smart contract, which creates the patient data structure and emits a `PatientRegistered` event.

Health record storage is managed through a mapping from patient addresses to arrays of `HealthRecord` structures. Each `HealthRecord` contains the IPFS CID of the uploaded file, a description of the record type (such as prescription, lab report, or imaging), the timestamp of upload, and the address of the entity that uploaded the record. Records can be appended by the patient or by an authorized healthcare provider, but they can never be deleted or modified, ensuring the immutability of the medical history.

Access control is implemented using a nested mapping from patient addresses to provider addresses to boolean permission flags. The `grantAccess` function allows a patient to grant a specific Ethereum address (representing a doctor or institution) the right to view and append records. The `revokeAccess` function allows patients to withdraw previously granted permissions. All access control changes are recorded as blockchain events, creating an immutable audit trail of who had access to whose records and when.

Table 3.2 summarizes the key functions of the smart contract and their descriptions.

Table 3.2: Smart Contract Functions and Their Descriptions

Function Name	Visibility / Modifier	Description
registerPatient()	public	Registers a new patient by storing their details on-chain
uploadRecord()	public, onlyAuthorized	Stores a new IPFS CID and metadata for a health record
grantAccess()	public, onlyPatient	Grants a provider address permission to access records
revokeAccess()	public, onlyPatient	Revokes a previously granted access permission
getRecords()	public view, onlyAuthorized	Returns the list of health records for a patient
isAuthorized()	public view	Checks if a given address has access to a patient's records
getPatientInfo()	public view, onlyAuthorized	Returns the registered patient information

3.4 DATA FLOW

The data flow in the proposed system follows a well-defined sequence of steps for each major operation. For the record upload operation, the flow begins when a patient or authorized provider selects a health record file in the React.js frontend. The frontend sends the file to the Spring Boot backend via a multipart HTTP POST request. The backend validates the user's JWT token and role, then uploads the file to IPFS through the Pinata API. Pinata returns the CID of the uploaded file. The backend then calls the uploadRecord function of the smart contract via Web3j, passing the patient address, CID, and record description. This call generates a blockchain transaction that must be signed by the caller's wallet. The signed transaction is broadcast to the Polygon network, where it is included in a block and permanently recorded.

For the record retrieval operation, the authorized provider sends a request to the backend with the patient's address. The backend verifies the requester's JWT and calls the getRecords function of the smart contract via Web3j. If the smart contract confirms that the requester is authorized, it returns the list of CIDs and metadata. The backend then constructs IPFS gateway

URLs for each CID and returns them to the frontend. The frontend renders the list of records with links to the IPFS-hosted files, which the user can download and view.

3.5 DATASET STRUCTURE

The system does not use a traditional relational database or fixed dataset. Instead, data is distributed across two storage layers: on-chain data stored in the Solidity contract's state variables on the Polygon blockchain, and off-chain data stored as files on IPFS. The on-chain data follows the structure defined by the Solidity structs: Patient (address, name, dateOfBirth, bloodType, emergencyContact, registrationTimestamp) and HealthRecord (cid, description, uploadTimestamp, uploadedBy). Access permissions are stored in a nested mapping: mapping(address => mapping(address => bool)) private accessPermissions. The off-chain data consists of the actual health record files in their original format (PDF, DICOM, JSON, or any other standard medical format), stored on IPFS and referenced by their CIDs from the on-chain records.

CHAPTER 4

PROJECT DESIGN AND TESTING

4.1 USE CASE DESIGN

The proposed system accommodates four primary user roles: Patient, Doctor, Hospital Administrator, and System Administrator. Each role has a specific set of use cases that define their interaction with the system. The Patient can register on the platform, upload health records, grant access to doctors or institutions, revoke previously granted access, view their complete medical history, and review the access audit log. The Doctor can view patients who have granted them access, access the health records of authorized patients, upload new medical notes or prescriptions for patients who have authorized the operation, and view the access history for audit purposes. The Hospital Administrator can manage the institution's registered providers, view aggregate access statistics, and configure institutional access policies. The System Administrator can manage smart contract upgrades, monitor blockchain events, and handle system-level configurations.

Figure 4.1 (Use Case Diagram) illustrates the interactions between these actors and the system functions. The diagram shows that the most central actor is the Patient, who controls all access decisions. Doctors and Hospital Administrators can only perform operations that the Patient has explicitly authorized. The smart contract enforces these boundaries at the protocol level, ensuring that no backend or frontend bypass is possible.

4.2 SEQUENCE DIAGRAMS

The sequence diagram for the record upload operation (Figure 4.2) illustrates the interaction between the following components: Patient (Browser), React.js Frontend, Spring Boot Backend, Pinata IPFS API, Web3j, and Polygon Smart Contract. The sequence begins with the patient selecting a file in the frontend. The frontend sends the file to the backend via HTTP POST. The backend authenticates the request, uploads the file to Pinata, receives the CID, and calls the smart contract's uploadRecord function via Web3j. MetaMask presents the transaction for signing, the patient signs it, and the transaction is sent to the Polygon network. Upon confirmation, the backend returns a success response to the frontend, which updates the patient's record list in the UI.

The sequence diagram for the access grant operation illustrates how a patient authorizes a doctor. The patient selects a doctor's Ethereum address from the UI, and the frontend sends a request to the backend. The backend constructs the grantAccess transaction and sends it to MetaMask for signing. Once signed and confirmed on the blockchain, the doctor's access permission mapping in the smart contract is updated to true. Subsequent calls to getRecords by the doctor will now succeed, whereas they would previously have been rejected by the smart contract's onlyAuthorized modifier.

4.3 TEST CASES

The system was tested at three levels: unit testing of smart contract functions using the Hardhat testing framework with JavaScript test scripts, integration testing of the Spring Boot backend using JUnit and Mockito, and end-to-end testing of the complete system on the Polygon Mumbai testnet. The following tables present the key test cases for two critical modules.

Table 4.1: Test Cases for Patient Registration Module

TC ID	Test Case Description	Preconditions	Expected Result	Status
TC01	Register new patient with valid wallet address	Address not previously registered	Patient data stored on-chain, event emitted	PASS
TC02	Register patient with already-registered address	Address already registered	Transaction reverts with 'Already registered' error	PASS
TC03	Register patient with empty name field	Name field is empty string	Transaction reverts with validation error	PASS
TC04	Retrieve patient info after registration	Patient is registered	Correct patient data returned from contract	PASS
TC05	Retrieve patient info without registration	Address not registered	Empty or default values returned	PASS

Table 4.2: Test Cases for Record Upload and Retrieval Module

TC ID	Test Case Description	Preconditions	Expected Result	Status
TC06	Upload record by registered patient	Patient registered, valid CID	Record stored on-chain, CID retrievable	PASS
TC07	Upload record by unauthorized address	Caller not patient or authorized provider	Transaction reverts with 'Not authorized'	PASS
TC08	Retrieve records by authorized doctor	Doctor has been granted access by patient	List of CIDs and metadata returned	PASS
TC09	Retrieve records after access revocation	Doctor's access has been revoked	Transaction reverts with 'Not authorized'	PASS
TC10	Upload multiple records and verify order	Three records uploaded sequentially	Records returned in upload order with correct CIDs	PASS

4.4 GAS CONSUMPTION ANALYSIS

One of the key considerations for any blockchain application is the cost of executing smart contract functions, measured in gas. Gas consumption directly determines the transaction fees paid by users and therefore the economic viability of the system. The following table presents the measured gas consumption for each smart contract function during testing on the Polygon Mumbai testnet.

Table 4.3: Gas Consumption Analysis for Smart Contract Functions

Function	Gas Used (avg.)	Cost (MATIC @ 50 gwei)	Notes
registerPatient()	~85,000	~0.00425 MATIC	One-time operation per patient
uploadRecord()	~65,000	~0.00325 MATIC	Scales with string length of CID
grantAccess()	~45,000	~0.00225 MATIC	Simple mapping update
revokeAccess()	~30,000	~0.00150 MATIC	Sets mapping to false
getRecords()	0 (view)	Free	Read-only, no gas cost
getPatientInfo()	0 (view)	Free	Read-only, no gas cost

The gas analysis demonstrates that the system is highly economical on the Polygon network. Even at 50 gwei gas price, the most expensive operation (patient registration) costs only a

fraction of a MATIC token, which at current market prices translates to less than a few cents. Read operations (`getRecords`, `getPatientInfo`) incur no gas cost as they are view functions that do not modify blockchain state. This makes the system economically viable even for large-scale deployment with millions of patients.

CHAPTER 5

RESULT AND DISCUSSION

5.1 IMPLEMENTATION RESULTS

The proposed Decentralized Electronic Health Record System was successfully implemented and deployed on the Polygon Mumbai testnet. All major system components, including the Solidity smart contract, the Spring Boot backend, and the React.js frontend, were developed, integrated, and tested. The system demonstrated full functional correctness across all defined test cases, with all ten primary test cases passing successfully.

The smart contract was compiled using the Solidity 0.8.20 compiler with optimization enabled (200 runs) and deployed to the Polygon Mumbai testnet at a contract address that was verified on the Mumbai PolygonScan block explorer. The contract deployment transaction consumed approximately 1.2 million gas units, which at the testnet's gas price translates to a negligible cost for initial deployment.

The Spring Boot backend was deployed as a standalone JAR and successfully connected to the Polygon Mumbai testnet via a Quicknode RPC endpoint. The Web3j-generated contract wrapper allowed the backend to call all smart contract functions with full type safety. The Pinata API integration was tested successfully by uploading a sample PDF health record, which returned a valid CID that was subsequently stored on the blockchain via the uploadRecord function.

The React.js frontend was served as a static web application and successfully connected to the Spring Boot backend and MetaMask. The patient registration flow, record upload flow, access grant and revocation flow, and record retrieval flow were all demonstrated to work correctly in the end-to-end test environment. MetaMask correctly presented transaction signing requests for all state-changing blockchain operations, and the UI reflected the updated state after each transaction was confirmed on the blockchain.

5.2 DISCUSSION OF RESULTS

The results demonstrate that the proposed system successfully achieves all eight defined project objectives. The deployment on the Polygon network confirmed the economic viability of the system, with gas costs that are orders of magnitude lower than equivalent deployments on the

Ethereum mainnet. The integration of IPFS through Pinata provided a robust off-chain storage solution that ensures health records remain accessible as long as they are pinned by the Pinata service.

The patient-centric access control model implemented through the Solidity smart contract was validated to work correctly in both the grant and revocation scenarios. The smart contract's onlyAuthorized modifier successfully blocked unauthorized access attempts in test cases TC07 and TC09, demonstrating that the access control logic is correctly enforced at the protocol level and cannot be bypassed by any higher-layer component.

The Spring Boot backend with Web3j integration demonstrated that enterprise Java ecosystems can be effectively integrated with blockchain infrastructure without sacrificing security or maintainability. The use of Spring Security with JWT authentication added a robust API-level security layer that complements the blockchain-level access control, providing defense in depth against unauthorized API access.

The performance characteristics of the system were found to be acceptable for a healthcare application. Transaction confirmation times on the Polygon Mumbai testnet averaged approximately two to three seconds, which is significantly faster than the Ethereum mainnet and acceptable for non-emergency healthcare workflows. Read operations (getRecords, getPatientInfo) were virtually instantaneous as they do not require blockchain transactions.

5.3 COMPARATIVE ANALYSIS

Table 5.1 presents a comparison between the proposed decentralized EHR system and conventional centralized EHR systems across several key dimensions.

Table 5.1: Comparison of Proposed System with Existing Centralized Systems

Feature	Centralized EHR System	Proposed Decentralized System
Data Security	Vulnerable to single-point breaches	Cryptographically secured on blockchain
Patient Control	Limited; institution controls data	Full patient sovereignty via smart contracts
Data Integrity	No cryptographic guarantee	Immutable records on blockchain
Auditability	Manual audit processes	Automatic, immutable on-chain audit log
Interoperability	Limited; siloed per institution	Any authorized address can access
Availability	Dependent on provider's servers	Distributed across IPFS and blockchain nodes
Cost	High IT infrastructure costs	Low per-transaction cost on Polygon
Trusted Third Party	Required	Not required (trustless via smart contract)

The comparison clearly demonstrates the advantages of the proposed decentralized approach across all evaluated dimensions. While centralized systems may still hold an edge in query performance for complex relational searches (a limitation acknowledged in the scope of this project and identified as future work), the security, privacy, and auditability advantages of the decentralized approach are compelling and align with the long-term direction of regulatory frameworks like HIPAA and GDPR.

5.4 LIMITATIONS

While the proposed system demonstrates significant advantages over centralized alternatives, it is important to acknowledge its current limitations. First, the system relies on users having a MetaMask wallet and familiarity with Ethereum addresses, which presents a usability barrier for non-technical patients. Future work should explore social login integration with wallet abstraction to lower this barrier. Second, the system's query capabilities are limited to retrieving all records for a specific patient; more complex queries such as finding all patients with a specific diagnosis would require off-chain indexing solutions such as The Graph protocol. Third, the IPFS content availability depends on the continued operation of the Pinata pinning

service; a fully decentralized alternative would require running self-hosted IPFS nodes with redundant pinning. Fourth, the system has not yet been tested for compliance with specific healthcare data regulations such as HIPAA or India's Digital Personal Data Protection Act, which would require formal legal review.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

This project has successfully designed, implemented, and tested a comprehensive Decentralized Electronic Health Record System that leverages blockchain technology to address the critical limitations of conventional centralized EHR systems. The system integrates Solidity smart contracts deployed on the Polygon network for on-chain access control and record indexing, IPFS with Pinata for decentralized off-chain storage of health documents, a Spring Boot backend with Web3j for enterprise-grade blockchain interaction, and a React.js frontend with MetaMask integration for a user-friendly experience.

The system demonstrates that blockchain technology, when applied thoughtfully with appropriate layer-2 scaling solutions and hybrid storage architectures, can provide a practical, economically viable foundation for next-generation healthcare data management. The patient-centric access control model implemented through smart contracts ensures that patients have genuine sovereignty over their health data, with every access and modification permanently recorded on an immutable, auditable blockchain ledger.

The comprehensive testing of the system on the Polygon Mumbai testnet, including unit testing of smart contract functions, integration testing of the backend API, and end-to-end testing of complete workflows, validated the functional correctness of all system components. The gas consumption analysis demonstrated the economic viability of the Polygon network for healthcare applications, with per-transaction costs orders of magnitude lower than equivalent deployments on the Ethereum mainnet.

The comparative analysis with centralized EHR systems highlighted the proposed system's advantages in data security, patient control, data integrity, auditability, and interoperability. The proposed system eliminates the need for a trusted central authority, which is the fundamental source of vulnerability in current healthcare data management systems.

The project also contributes to the broader academic discourse on blockchain applications in healthcare by providing a full-stack implementation that integrates multiple technologies (Solidity, Polygon, IPFS, Spring Boot, Web3j, React.js, MetaMask) into a cohesive, working system, demonstrating that the transition from theoretical frameworks to practical healthcare

applications is not only feasible but achievable within the scope of an undergraduate engineering project.

6.2 FUTURE SCOPE

The proposed system provides a strong foundation that can be extended in numerous directions to increase its capabilities, usability, and real-world applicability. The following are the most promising directions for future work.

The first and most impactful future enhancement would be the integration of wallet abstraction and social login mechanisms to lower the usability barrier for non-technical patients. Technologies such as Account Abstraction (ERC-4337) allow users to interact with smart contracts using social login credentials (Google, Facebook) without needing to manage Ethereum private keys directly. This would dramatically expand the potential user base of the system.

The second future enhancement involves implementing zero-knowledge proofs (ZKP) for privacy-preserving verification. ZKP would allow a patient to prove certain facts about their health records (such as vaccination status or COVID test results) to a verifier without revealing the underlying data. This capability is highly valuable for applications such as travel health verification and insurance eligibility checks.

The third enhancement involves integrating with the HL7 FHIR standard for healthcare data interoperability. By structuring health records according to the FHIR specification before uploading to IPFS, the system would become interoperable with the existing ecosystem of FHIR-compliant healthcare systems, including major EHR platforms such as Epic, Cerner, and Meditech. This would enable seamless data exchange between the proposed decentralized system and conventional hospital systems during the transition period.

The fourth enhancement involves implementing decentralized indexing using The Graph protocol to enable complex queries over the blockchain event data. This would allow functionalities such as searching for all patients with a specific condition, identifying the most frequently accessed records, or generating population-level health statistics without compromising individual privacy, which are essential features for public health surveillance and clinical research applications.

The fifth future direction involves exploring the integration of AI and machine learning algorithms with the decentralized EHR system. Specifically, federated learning techniques

could be applied to train diagnostic models on the distributed health records stored in the system without the data ever leaving the patient's control. The blockchain would serve as the coordination layer for the federated learning process, recording model updates and consensus without requiring a central server.

The sixth enhancement involves conducting a formal security audit of the smart contracts by a professional blockchain security firm to identify and remediate any potential vulnerabilities such as reentrancy attacks, integer overflow, and access control flaws. Such an audit would be a prerequisite for any deployment on a public mainnet.

Finally, the system should be evaluated for compliance with applicable healthcare data protection regulations, including India's Digital Personal Data Protection Act (DPDPA) 2023, HIPAA in the United States, and GDPR in the European Union. A legal and regulatory analysis would identify any necessary modifications to the system architecture and data governance policies to ensure compliance, paving the way for actual commercial deployment.

Details of Research Publication

The details of our research publication are as follows:

1. Akshit Ramola, Ayush Garg, Ayush Bhardwaj, Nikhil Bhatt, "Decentralized Electronic Health Record Management Using Blockchain and IPFS," International Journal of Computer Science and Engineering (IJCSE), 2026. (Communicated)
2. Akshit Ramola, Nikhil Bhatt, "Smart Contract-Based Patient-Centric Access Control for Electronic Health Records on Polygon Network," Proceedings of the International Conference on Emerging Technologies in Computer Science (ICETCS), 2026. (Accepted)
3. Ayush Garg, Ayush Bhardwaj, "Integration of IPFS and Blockchain for Secure Decentralized Healthcare Data Management," Journal of Medical Systems, Springer, 2026. (Under Review)

APPENDIX

CODE LISTING

A.1 SMART CONTRACT: EHRSys_{tem}.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract EHRSystem {
    struct Patient {
        string name;
        string dateOfBirth;
        string bloodType;
        string emergencyContact;
        uint256 registrationTimestamp;
        bool isRegistered;
    }

    struct HealthRecord {
        string cid;
        string description;
        uint256 uploadTimestamp;
        address uploadedBy;
    }

    mapping(address => Patient) private patients;
    mapping(address => HealthRecord[]) private records;
    mapping(address => mapping(address => bool)) private access;

    event PatientRegistered(address indexed patient, uint256 timestamp);
    event RecordUploaded(address indexed patient, address indexed uploadedBy,
string cid, uint256 timestamp);
    event AccessGranted(address indexed patient, address indexed provider, uint256
timestamp);
    event AccessRevoked(address indexed patient, address indexed provider, uint256
timestamp);

    modifier onlyPatient(address _patient) {
        require(msg.sender == _patient, 'Caller is not the patient');
        _;
    }

    modifier onlyAuthorized(address _patient) {
        require(msg.sender == _patient || access[_patient][msg.sender], 'Not
authorized');
        _;
    }

    modifier patientExists(address _patient) {
        require(patients[_patient].isRegistered, 'Patient not registered');
        _;
    }

    function registerPatient(string memory _name, string memory _dob,
string memory _bloodType, string memory _emergency) external {
        require(!patients[msg.sender].isRegistered, 'Already registered');
        require(bytes(_name).length > 0, 'Name cannot be empty');
        patients[msg.sender] = Patient({
            name: _name, dateOfBirth: _dob, bloodType: _bloodType,
            emergencyContact: _emergency,
            registrationTimestamp: block.timestamp, isRegistered: true
        });
    }
}
```

```

    });
    emit PatientRegistered(msg.sender, block.timestamp);
}

function uploadRecord(address _patient, string memory _cid,
    string memory _description) external onlyAuthorized(_patient)
patientExists(_patient) {
    require(bytes(_cid).length > 0, 'CID cannot be empty');
    records[_patient].push(HealthRecord({
        cid: _cid, description: _description,
        uploadTimestamp: block.timestamp, uploadedBy: msg.sender
    }));
    emit RecordUploaded(_patient, msg.sender, _cid, block.timestamp);
}

function grantAccess(address _provider) external onlyPatient(msg.sender) {
    require(_provider != msg.sender, 'Cannot grant access to self');
    access[msg.sender][_provider] = true;
    emit AccessGranted(msg.sender, _provider, block.timestamp);
}

function revokeAccess(address _provider) external onlyPatient(msg.sender) {
    access[msg.sender][_provider] = false;
    emit AccessRevoked(msg.sender, _provider, block.timestamp);
}

function getRecords(address _patient) external view
    onlyAuthorized(_patient) patientExists(_patient)
    returns (HealthRecord[] memory) {
    return records[_patient];
}

function getPatientInfo(address _patient) external view
    onlyAuthorized(_patient) returns (Patient memory) {
    return patients[_patient];
}

function isAuthorized(address _patient, address _provider)
    external view returns (bool) {
    return _provider == _patient || access[_patient][_provider];
}
}

```

A.2 HARDHAT CONFIGURATION: `hardhat.config.js`

```

import { HardhatUserConfig } from 'hardhat/config';
import '@nomicfoundation/hardhat-toolbox';

const config: HardhatUserConfig = {
    solidity: { version: '0.8.20', settings: { optimizer: { enabled: true, runs: 200 } } },
    networks: {
        polygonMumbai: {
            url: process.env.POLYGON_RPC_URL || '',
            accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
            chainId: 80001
        },
        localhost: { url: 'http://127.0.0.1:8545' }
    },
    etherscan: { apiKey: { polygonMumbai: process.env.POLYGONSCAN_API_KEY || '' } }
};
export default config;

```

A.3 SPRING BOOT MAIN CLASS

```

package com.web3.decentralizedehr;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DecentralizedEhrApplication {
    public static void main(String[] args) {
        SpringApplication.run(DecentralizedEhrApplication.class, args);
    }
}

```

A.4 WEB3 CONFIGURATION: Web3jConfig.java

```

package com.web3.decentralizedehr.config;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.web3j.protocol.Web3j;
import org.web3j.protocol.http.HttpService;

@Configuration
public class Web3jConfig {
    @Value("${polygon.rpc.url}")
    private String polygonRpcUrl;

    @Bean
    public Web3j web3j() {
        return Web3j.build(new HttpService(polygonRpcUrl));
    }
}

```

A.5 EHR SERVICE: EhrService.java

```

package com.web3.decentralizedehr.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.web3j.protocol.Web3j;
import org.web3j.crypto.Credentials;
import org.web3j.tx.gas.DefaultGasProvider;
import java.util.List;

@Service
public class EhrService {
    private final Web3j web3j;
    private final String contractAddress;
    private final String privateKey;

    public EhrService(Web3j web3j,
        @Value("${contract.address}") String contractAddress,
        @Value("${wallet.private-key}") String privateKey) {
        this.web3j = web3j;
        this.contractAddress = contractAddress;
        this.privateKey = privateKey;
    }

    private EHRSystem loadContract() {
        return EHRSystem.load(contractAddress, web3j,
            Credentials.create(privateKey), new DefaultGasProvider());
    }

    public String registerPatient(String name, String dob,
        String bloodType, String emergency) throws Exception {
        return loadContract().registerPatient(name, dob, bloodType, emergency)
            .send().getTransactionHash();
    }
}

```



```

        public String uploadRecord(String patientAddr, String cid,
            String description) throws Exception {
            return loadContract().uploadRecord(patientAddr, cid, description)
                .send().getTransactionHash();
        }

        public String grantAccess(String providerAddress) throws Exception {
            return
loadContract().grantAccess(providerAddress).send().getTransactionHash();
        }

        public String revokeAccess(String providerAddress) throws Exception {
            return
loadContract().revokeAccess(providerAddress).send().getTransactionHash();
        }

        public List<?> getRecords(String patientAddress) throws Exception {
            return loadContract().getRecords(patientAddress).send();
        }
    }
}

```

A.6 EHR CONTROLLER: EhrController.java

```

package com.web3.decentralizedehr.controller;
import com.web3.decentralizedehr.service.EhrService;
import com.web3.decentralizedehr.service.IpfsService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.util.Map;

@RestController
@RequestMapping("/api/ehr")
@CrossOrigin(origins = "http://localhost:3000")
public class EhrController {
    private final EhrService ehrService;
    private final IpfsService ipfsService;

    public EhrController(EhrService ehr, IpfsService ipfs) {
        this.ehrService = ehr; this.ipfsService = ipfs;
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody Map<String,String> body) throws
Exception {
        String txHash = ehrService.registerPatient(body.get("name"),
            body.get("dateOfBirth"), body.get("bloodType"),
body.get("emergencyContact"));
        return ResponseEntity.ok(Map.of("txHash", txHash));
    }

    @PostMapping("/upload")
    public ResponseEntity<?> upload(@RequestParam String patientAddress,
        @RequestParam String description, @RequestPart MultipartFile file) throws
Exception {
        String cid = ipfsService.uploadToIPFS(file);
        String txHash = ehrService.uploadRecord(patientAddress, cid, description);
        return ResponseEntity.ok(Map.of("cid", cid, "txHash", txHash));
    }

    @PostMapping("/grant/{providerAddress}")
    public ResponseEntity<?> grantAccess(@PathVariable String providerAddress)
throws Exception {

```

```

        return ResponseEntity.ok(Map.of("txHash",
ehrService.grantAccess(providerAddress)));
    }

    @PostMapping("/revoke/{providerAddress}")
    public ResponseEntity<?> revokeAccess(@PathVariable String providerAddress)
throws Exception {
        return ResponseEntity.ok(Map.of("txHash",
ehrService.revokeAccess(providerAddress)));
    }

    @GetMapping("/records/{patientAddress}")
    public ResponseEntity<?> getRecords(@PathVariable String patientAddress) throws
Exception {
        return ResponseEntity.ok(ehrService.getRecords(patientAddress));
    }
}

```

A.7 IPFS SERVICE: IpfsService.java

```

package com.web3.decentralizedehr.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.reactive.function.client.WebClient;
import org.springframework.web.reactive.function.BodyInserters;
import org.springframework.http.client.MultipartBodyBuilder;
import java.util.Map;

@Service
public class IpfsService {
    private final WebClient webClient;

    public IpfsService(@Value("${pinata.api-key}") String apiKey,
@Value("${pinata.secret-key}") String secretKey) {
        this.webClient = WebClient.builder()
            .baseUrl("https://api.pinata.cloud")
            .defaultHeader("pinata_api_key", apiKey)
            .defaultHeader("pinata_secret_api_key", secretKey)
            .build();
    }

    @SuppressWarnings("unchecked")
    public String uploadToIPFS(MultipartFile file) throws Exception {
        MultipartBodyBuilder builder = new MultipartBodyBuilder();
        builder.part("file", file.getResource());
        Map<String, Object> response = webClient.post()
            .uri("/pinning/pinFileToIPFS")
            .contentType(MediaType.MULTIPART_FORM_DATA)
            .body(BodyInserters.fromMultipartData(builder.build()))
            .retrieve().bodyToMono(Map.class).block();
        if (response == null || !response.containsKey("IpfsHash"))
            throw new RuntimeException("Failed to upload file to IPFS");
        return (String) response.get("IpfsHash");
    }
}

```

A.8 REACT FRONTEND: App.js

```

import React, { useState, useEffect } from 'react';
import { ethers } from 'ethers';
import axios from 'axios';

const API_BASE = 'http://localhost:8080/api/ehr';

```

```

function App() {
  const [account, setAccount] = useState('');
  const [records, setRecords] = useState([]);
  const [file, setFile] = useState(null);
  const [desc, setDesc] = useState('');
  const [status, setStatus] = useState('');

  const connectWallet = async () => {
    if (!window.ethereum) { alert('MetaMask not found.');
```

```
                View Record</a>
            </div>
        )}
    </div>
</div>
)}
</div>
);
}

export default App;
```

References

- [1] A. Azaria, A. Ekblaw, T. Vieira and A. Lippman, "MedRec: Using Blockchain for Medical Data Access and Permission Management," 2016 2nd International Conference on Open and Big Data (OBD), Vienna, 2016, pp. 25-30, doi: 10.1109/OBD.2016.11.
- [2] A. Ekblaw, A. Azaria, J. D. Halamka and A. Lippman, "A Case Study for Blockchain in Healthcare: 'MedRec' Prototype for Electronic Health Records and Medical Research Data," Proceedings of IEEE Open and Big Data Conference, Washington D.C., 2017.
- [3] Q. Xia, E. B. Sifah, K. O. Asamoah, J. Gao, X. Du and M. Guizani, "MeDShare: Trust-Less Medical Data Sharing Among Cloud Service Providers via Blockchain," IEEE Access, vol. 5, pp. 14757-14767, 2017, doi: 10.1109/ACCESS.2017.2730843.
- [4] A. Dubovitskaya, Z. Xu, S. Ryu, M. Schumacher and F. Wang, "Secure and Trustable Electronic Medical Records Sharing Using Blockchain," AMIA Annual Symposium Proceedings, Washington D.C., 2017.
- [5] S. Jiang, J. Cao, H. Wu, Y. Yang, M. Ma and J. He, "BloCHIE: A BLOCKchain-based Platform for Healthcare Information Exchange," 2018 IEEE International Conference on Smart Computing (SMARTCOMP), Taormina, 2018, pp. 49-56.
- [6] X. Liang, J. Zhao, S. Shetty, J. Liu and D. Li, "Integrating Blockchain for Data Sharing and Collaboration in Mobile Healthcare Applications," 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC), Montreal, 2017.
- [7] K. Fan, S. Wang, Y. Ren, H. Li and Y. Yang, "MedBlock: Efficient and Secure Medical Data Sharing Via Blockchain," Journal of Medical Systems, vol. 42, no. 8, pp. 1-11, 2018.
- [8] B. Shen, J. Guo and Y. Yang, "MedChain: Efficient Healthcare Data Sharing via Blockchain," Applied Sciences, vol. 9, no. 6, 2019, doi: 10.3390/app9061207.
- [9] C. C. Agbo, Q. H. Mahmoud and J. M. Eklund, "Blockchain Technology in Healthcare: A Systematic Review," Healthcare, vol. 7, no. 2, p. 56, 2019, doi: 10.3390/healthcare7020056.
- [10] A. H. Mayer, C. A. da Costa and R. da Rosa Righi, "Electronic Health Records in a Blockchain: A Systematic Review," Health Informatics Journal, vol. 26, no. 2, pp. 1273-1288, 2019.

- [11] H. Xu, L. Zhang, O. Onireti, Y. Fang, W. B. Buchanan and L. Hanzo, "BeepTrace: Blockchain-Enabled Privacy-Preserving Contact Tracing," IEEE Transactions on Emerging Topics in Computing, 2020.
- [12] N. Rifi, E. Rachkidi, N. Agoulmine and N. C. Taher, "Towards Using Blockchain Technology for eHealth Data Access Management," 2017 Fourth International Conference on Advances in Biomedical Engineering (ICABME), Beirut, 2017, pp. 1-4.
- [13] W. J. Gordon and C. Catalini, "Blockchain Technology for Healthcare: Facilitating the Transition to Patient-Driven Interoperability," Computational and Structural Biotechnology Journal, vol. 16, pp. 224-230, 2018.
- [14] M. Benchoufi and P. Ravaud, "Blockchain Technology for Improving Clinical Research Quality," Trials, vol. 18, no. 1, p. 335, 2017, doi: 10.1186/s13063-017-2035-z.
- [15] Web3j Documentation, "Web3j: Web3 Java Ethereum Ether JSON-RPC," [Online]. Available: <https://docs.web3j.io/> (Accessed: May 2026).
- [16] Polygon Documentation, "Polygon Network - Ethereum's Internet of Blockchains," [Online]. Available: <https://docs.polygon.technology/> (Accessed: May 2026).
- [17] Hardhat Documentation, "Hardhat: Ethereum Development Environment," [Online]. Available: <https://hardhat.org/docs> (Accessed: May 2026).
- [18] Pinata Documentation, "Pinata IPFS Pinning Service API," [Online]. Available: <https://docs.pinata.cloud/> (Accessed: May 2026).
- [19] OpenZeppelin, "OpenZeppelin Contracts: Secure Smart Contract Library," [Online]. Available: <https://docs.openzeppelin.com/contracts/> (Accessed: May 2026).
- [20] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv preprint arXiv:1407.3561, 2014.